

Footnote: AI-Powered Glossary Extension for Video/Podcast Content

The Problem

When listening to technical podcasts and video content (AI/ML, science, medicine, finance, etc.), speakers frequently use jargon, acronyms, and domain-specific concepts without defining them. The listener faces a bad tradeoff: either pause to Google every unfamiliar term (breaking flow) or keep listening and miss the substance. There is no equivalent of a "footnote" for audio/video content — proactive, contextual, inline definitions that appear as you need them.

The Idea

A Chrome extension that acts as a real-time glossary sidebar for YouTube videos. When you open a video, the extension scrapes the transcript, sends it to an LLM for analysis, and renders a scrollable sidebar of term cards — each containing a short definition and deeper explanation. As the video plays, the sidebar highlights and auto-scrolls to the term currently being discussed, like footnotes synced to playback.

The key differentiator from existing tools (like Snipd's "chat with episode" or generic summarizers) is that this is **proactive, not reactive**. The user doesn't have to stop and ask a question. Terms are pre-identified and ranked by likelihood the user doesn't already know them, based on a configurable listener profile.

Competitive Landscape

What exists today

- **Snipd** — AI podcast player with transcripts, summaries, chapters, and a "chat with episode" feature where you can ask questions about content after listening. Closest competitor, but reactive (user must type a question) and locked to their proprietary player. No proactive term detection.
- **AI PodNav** — Transcription, summarization, mind mapping for podcasts. Focused on high-level summaries, not term-level comprehension.
- **Shortcast AI** — Condenses long episodes into short AI-narrated summaries. Entirely different use case (time-saving vs. comprehension).
- **Generic ChatGPT/Claude usage** — Users can paste transcripts into an LLM and ask questions manually. Works but high-friction, no integration with playback, no timestamp sync.

The gap

Nobody is doing **proactive, timestamped, personalized glossary generation** for audio/video content. Existing tools either summarize (top-down) or answer questions (reactive). This project fills the "footnote" layer — term-level, bottom-up comprehension support that's automatic and synced to playback.

MVP Scope (Weekend Build)

What it is

A locally-running Chrome extension + Python backend that works on YouTube videos. Single user. No auth, no accounts, no cloud deployment. Designed to work on one person's machine as a proof-of-concept.

What it is NOT

- Not a standalone app or podcast player
- Not a summarizer or chatbot
- Not multi-user or deployed anywhere
- Not real-time streaming analysis (processes full transcript upfront)

Architecture

Two components

1. Chrome Extension (Manifest V3)

- Content script injected on youtube.com
- Scrapes the video transcript (via YouTube's timedtext endpoint or DOM transcript panel)
- Sends full transcript to local backend
- Renders a sidebar panel next to the video player
- Displays term cards sorted by timestamp
- Watches video playback position and highlights/auto-scrolls to the active term card
- "Dismiss" button on each card to mark terms as known (saved to local JSON for future prompt inclusion)

2. Local Python Backend (FastAPI)

- Single-file server running on localhost:8000
- Receives transcript text from the extension
- Calls the Anthropic API (Claude Sonnet 4.5) with a detailed extraction prompt
- Returns structured JSON: list of terms with definitions, categories, and transcript quotes

- Caches results to a local JSON file keyed by YouTube video ID (avoids re-processing the same video)
- Post-processes LLM output: deduplication, timestamp resolution via quote-matching

Data flow

```
YouTube page loaded
  → Content script scrapes transcript with timestamps
  → POST transcript to localhost:8000/extract
  → FastAPI handler checks cache (video_id.json)
    → Cache hit: return cached terms
    → Cache miss: call Claude API with extraction prompt
      → Parse JSON response
      → Deduplicate terms
      → Resolve timestamps by matching quotes to transcript lines
      → Cache result
      → Return terms
  → Extension receives terms JSON
  → Renders sidebar with term cards
  → Playback listener highlights active card as video plays
```

The Extraction Prompt (Core IP)

The prompt is the heart of the product. It instructs the LLM to:

1. **Identify technical terms, acronyms, and named concepts** that the speakers use without defining
2. **Filter based on a listener profile** — a configurable description of what the user already knows, so obvious terms are skipped and only genuinely useful ones surface
3. **Skip terms the speakers define themselves** — if the podcast already explains it, the listener doesn't need a footnote
4. **Target density of ~1 term per 3-5 minutes** — a 2-hour podcast should yield 20-40 terms, not 200
5. **Deduplicate** — same acronym and its expansion don't get separate entries; first occurrence only
6. **Return structured JSON** with fields:
 - `term` — as spoken in the transcript
 - `expansion` — full form if it's an acronym, null otherwise
 - `one_liner` — plain-language definition under 20 words

- `deeper` — 2-3 sentences of context, why it matters, how it relates to the conversation
- `quote` — verbatim 5-15 word phrase from the transcript (used for reliable timestamp matching via string search, rather than trusting the LLM to output correct timestamps)
- `category` — one of: `ml_research`, `biology`, `neuroscience`, `physics`, `economics`, `medicine`, `cs_systems`, `math_stats`, `named_entity`, `other`

Key Design Decisions

Why process the full transcript upfront (not streaming)?

YouTube pre-recorded videos already have the full transcript available. One-shot extraction is simpler, cheaper, and avoids the complexity of chunked processing, context windows, and streaming UI updates. The sidebar loads once when you open the video.

Why use quote-matching for timestamps instead of asking the LLM?

LLMs hallucinate timestamps. By asking Claude to return a verbatim quote and then doing a substring match against the timestamped transcript in Python, we get reliable timestamp resolution without trusting the model's counting ability.

Why a local backend instead of calling the API from the extension?

API keys shouldn't live in extension code (even locally). A local FastAPI server also gives a clean place for caching, dedup logic, and future features without touching the extension.

Why YouTube-only for MVP?

YouTube auto-generates captions for almost all content, has massive overlap with technical podcast/lecture content, and exposes transcripts in a scrapeable format. One platform, one transcript format, one DOM structure. Expand to Spotify web, Apple Podcasts web, or arbitrary audio files later.

Tech Stack

Component	Technology	Rationale
Extension	Plain JS, Manifest V3	No build step, fast iteration, minimal complexity
Backend	Python, FastAPI	Single file, fast to write, good Anthropic SDK support
LLM	Claude Sonnet 4.5 (extraction)	Best quality/cost tradeoff for nuanced term selection

Component	Technology	Rationale
Coding assistant	Claude Sonnet 4.6 via Claude Code	Sufficient for extension + FastAPI boilerplate, saves tokens vs Opus
Cache	Local JSON files	No database needed for single-user MVP
Known terms	Local JSON file	Persists dismissed terms across sessions

Future Directions (Post-MVP)

These are out of scope for the weekend but represent the product roadmap if the MVP validates:

- **Personalization model** — learn which terms the user dismisses over time, build a per-user "known vocabulary" that improves filtering
- **Cross-platform** — extend to Spotify web player, Apple Podcasts web, arbitrary audio file upload
- **Prerequisite chains** — "you don't know term A, but to understand it you also need term B" — dependency-aware explanations
- **Cross-episode continuity** — "you learned this term in episode 47 of Lex Fridman, here it is again with a new wrinkle"
- **Difficulty personas** — presets like "ML engineer", "bio-curious generalist", "complete beginner" that swap the listener profile
- **Haiku cost optimization** — test whether Claude Haiku 4.5 can match Sonnet's extraction quality for this specific task to reduce per-episode cost
- **Sharing/export** — export glossary as markdown, Anki cards, or Notion page

Suggested Build Order

When	Task	Why this order
Sat morning	Backend + prompt tuning	The prompt is the product. Get extraction quality right before building UI around it. Test with curl against 2-3 real transcripts.
Sat afternoon	Extension skeleton	Scrape transcript, POST to localhost, render raw cards in a sidebar. Ugly is fine.
Sunday morning	Playback sync	Highlight active card, auto-scroll sidebar as video plays.

When	Task	Why this order
Sunday afternoon	Dismiss button + polish	Wire up "I know this" to local JSON. Try on 5 real episodes. Iterate prompt based on what's missing or noisy.

Evaluation Criteria

To know if the MVP is working well:

1. Pick 3 podcast episodes you know well
2. Manually list the 10-15 terms you'd want footnoted in each
3. Run the extraction and measure:
 - **Precision** — what % of returned terms are actually useful?
 - **Recall** — what % of your manual list got caught?
 - **Quality** — are the definitions accurate and the right depth?
4. Target: >80% precision, >70% recall, zero hallucinated definitions